



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2413 — Bases de Datos — Sección 3 — 2'2026

TAREA 1

Tarea: Escritura de consultas.
Publicación: Lunes 24 de Agosto.
Entrega: **Jueves 3 de Septiembre a las 23:59 horas.**

Modalidad de trabajo

Esta tarea es de carácter **individual**. El desarrollo, la implementación y la redacción del informe deben ser realizados exclusivamente por cada estudiante. Cualquier intento de plagio o copia será sancionado de acuerdo con las normativas académicas de la Escuela de Ingeniería.

Objetivos de la tarea

1. Aprender a procesar datos desde archivos (CSV), crear tablas con SQL y poblar la base de datos.
2. Practicar el diseño, escritura y uso de consultas SQL en un contexto real.
3. Programar usando SQL con consultas parametrizadas a través de funciones en Python.

Contexto: Brazilian E-Commerce Public Dataset

En esta tarea trabajará con un conjunto de datos (dataset) real proporcionado por *Olist* (<https://olist.com/>), una plataforma brasileña para gestionar un comercio electrónico (estilo “marketplace”). Este dataset contiene información de más de cien mil órdenes de compra realizadas entre 2016 y 2018 en distintas plataformas.

Los datos proporcionados representan el ciclo completo de ventas de un comercio electrónico, teniendo las siguientes entidades lógicas representadas por relaciones (esto es, tablas):

- **Transacciones:** `orders` es el núcleo del modelo, donde cada entrada registra cronológicamente los eventos logísticos sobre las ventas (compra, aprobación del pago, envío y entrega).
- **Actores:** `customers` y `sellers` son las relaciones que representan a los usuarios de la plataforma (esto es, clientes y vendedores), conteniendo sus IDs y ubicación geográfica.
- **Catálogo y Finanzas:** `order_items` detalla los productos que forman parte de la orden y su costo de envío. `products` contiene la información específica por producto y `order_payments` incluye los métodos de pago, monto y cuotas.
- **Experiencia:** `order_reviews` contiene la puntuación y review del cliente sobre la compra.

Archivos del dataset de Olist

Para el desarrollo de esta tarea, contará con un archivo `.zip` el cual incluye un archivo `jupyter notebook` (`.ipynb`) y ocho archivos `.csv` (nueve archivos en total) dentro de una carpeta `/data`. La descripción de los archivos es la siguiente:

1. **T1_plantilla.ipynb:** Este notebook interactivo contiene todo lo necesario para la tarea. Con este archivo debe trabajar y debe subir para la entrega.
2. **olist_customers_dataset.csv:** Es el archivo que contiene toda la información sobre los compradores (`customers`), específicamente:
 - `customer_id`: ID del comprador que se genera por cada orden que se realiza.
 - `customer_unique_id`: ID ÚNICO del comprador. Este es el identificador real, si un usuario realiza 3 compras, tendrá 3 `customer_id` distintos, pero siempre el mismo `customer_unique_id`.
 - `customer_zip_code_prefix`: Prefijo del código postal del comprador.
 - `customer_city`: Nombre de la ciudad del comprador.
 - `customer_state`: Sigla del estado de residencia del comprador (ej: SP para Sao Paulo, RS para Rio grande do Sul).
3. **olist_order_items_dataset.csv:** Es el archivo que contiene el detalle de la compra (`order_items`), específicamente:
 - `order_id`: ID único por cada orden.
 - `order_item_id`: Número secuencial (1, 2, 3, ...) que cuenta la cantidad de ítems en la misma orden.
 - `product_id`: ID del producto.
 - `seller_id`: ID del vendedor, una misma orden puede tener más de un `seller_id`, ya que Olist es un tipo marketplace, podemos realizar una sola compra a varios vendedores (como en Mercado Libre o Aliexpress).
 - `shipping_limit_date`: Fecha límite que tiene el vendedor para entregar el paquete a la empresa de envío.
 - `price`: Precio unitario por producto.
 - `freight_value`: Valor del envío correspondiente a cada ítem de manera individual. Por ejemplo, si una compra incluye dos productos, el costo total del envío se obtiene sumando el `freight_value` de cada uno de ellos (y de igual forma se extrapola para tres o más ítems).
4. **olist_order_payments_dataset.csv:** Es el archivo que contiene la información sobre el pago de las órdenes (`order_payments`), específicamente:
 - `order_id`: ID único por cada orden.
 - `payment_sequential`: El número de secuencia del pago para la orden, si el cliente pagó en un solo proceso, será 1, si pagó con una gift-card y luego con tarjeta de crédito, serían dos filas con valores 1 y 2.
 - `payment_type`: Método de pago.
 - `payment_installments`: Cantidad de cuotas.
 - `payment_value`: Monto exacto del pago durante esa secuencia.
5. **olist_order_reviews_dataset.csv:** Es el archivo que contiene la información sobre las reviews de las órdenes (`order_reviews`), específicamente:

- **review_id**: ID único de la review.
 - **order_id**: ID único por cada orden.
 - **review_score**: Cuántas estrellas dieron (de 1 a 5).
 - **review_comment_title**: Título de la review.
 - **review_comment_message**: Contenido de la review.
 - **review_creation_date**: Fecha en la que Olist generó la encuesta al cliente (notar que NO es cuando se publicó la review).
 - **review_answer_timestamp**: Fecha y hora en la que el comprador efectivamente publicó la review.
6. **olist_orders_dataset.csv**: Es el archivo que contiene la información sobre las órdenes de compra (**orders**), específicamente:
- **order_id**: ID único por cada orden.
 - **customer_id**: ID del comprador, este se genera por cada orden que se realiza.
 - **order_status**: Estado de la compra.
 - **order_purchase_timestamp**: Fecha y hora en la que se confirma el pedido y se genera la compra en el sistema.
 - **order_approved_at**: Fecha y hora en la que el pago fue aprobado.
 - **order_delivered_carrier_date**: Fecha en la que el vendedor entregó el paquete al envío.
 - **order_delivered_customer_date**: Fecha en la que el cliente recibe el paquete.
 - **order_estimated_delivery_date**: Fecha límite en la que Olist promete el recibo del paquete.
7. **olist_products_dataset.csv**: Es el archivo que contiene la información sobre los productos (**products**), específicamente:
- **product_id**: ID del producto.
 - **product_category_name**: Nombre de la categoría del producto, donde este nombre está en portugués.
 - **product_name_length**: Largo del nombre del producto.
 - **product_description_length**: Largo de la descripción del producto.
 - **product_photos_qty**: Cantidad de fotos que tiene el producto.
 - **product_weight_g**: Peso total del producto en gramos.
 - **product_length_cm**: Largo total del producto en centímetros.
 - **product_height_cm**: Altura total del producto en centímetros.
 - **product_width_cm**: Anchura total del producto en centímetros.
8. **olist_sellers_dataset.csv**: Es el archivo que contiene la información sobre los vendedores (**sellers**), específicamente:
- **seller_id**: ID del vendedor.
 - **seller_zip_code_prefix**: Prefijo del código postal del vendedor.
 - **seller_city**: Nombre de la ciudad del vendedor.
 - **seller_state**: Sigla del estado de residencia del vendedor (ej: SP para Sao Paulo, RS para Rio grande do Sul).
9. **product_category_name_translation.csv**: Es el archivo que contiene la información sobre la traducción de las categorías desde el portugués al inglés (**translation**), específicamente:
- **product_category_name**: Nombre de la categoría del producto en portugués.
 - **product_category_name_english**: Nombre de la categoría del producto en inglés.

SQLite y librerías

En esta tarea ocuparemos el sistema de bases de datos SQLite, que es un motor ligero y que no necesita de un servicio, lo cual es ideal para una primera experiencia con este tipo de tecnología. Por lo tanto, para el desarrollo de esta tarea, trabajará con las siguientes tres librerías:

- **sqlite3**: librería para ejecutar y acceder SQLite.
- **pandas**: librería para manejar y analizar datos.
- **numpy**: librería para funciones matemáticas y trabajo de datos.

sqlite3 viene incluido por defecto en la biblioteca estándar de Python. Por lo tanto, ni la librería ni el motor de bases de datos requieren instalación. En cambio, para instalar **pandas** y **numpy** basta con ejecutar el siguiente comando: `pip install pandas numpy`.

Problemas a resolver

La tarea está subdividida en **cuatro partes** que usted debe resolver.

Parte 1: Creación de base de datos (1.25 Puntos)

Esta primera parte se enfoca en comprender cómo procesar información, crear tablas SQL y realizar la inserción de los datos. En el notebook se encuentra la siguiente función:

```
def setup_database(db_path):  
    pass
```

Esta función será la encargada de generar la base de datos, donde “db_path” es el nombre del archivo final que tendrá nuestra base de datos. Para crear y poblar la base de datos, usted debe cumplir con los siguientes seis pasos:

1. **Cargar los csv:** Usando **pandas**, debe cargar los archivos CSV en memoria.
2. **Transformar fechas:** SQLite no soporta el tipo de dato DATETIME, por lo que debe transformarlos a strings, según este formato: “%Y-%m-%d %H:%M:%S”

```
df[col] = pd.to_datetime(df[col], errors='coerce').dt.strftime('%Y-%m-%d %H:%M:%S')
```

donde df y col son el dataframe y columna específica donde se realiza el cambio.

3. **Rellenado de nulos:** Si una review no tiene título, debe rellenar con “Sin título”, si no tiene contenido la review, se debe rellenar con “Sin mensaje”, si el nombre de la categoría de un producto está vacío, debe rellenarse con “desconocido”, si el largo del nombre del producto está vacío, se debe rellenar con 0, mismo caso para la descripción y las fotos, finalmente todos los valores NaN se deben reemplazar por None.
4. **Reemplazo de NaN:** Tiene que reemplazar todos los valores NaN de todos los dataframes por None, esto se consigue con:

```
df.replace({np.nan: None}, inplace=True)
```

con df el dataframe específico en el que se realiza el reemplazo.

5. **Crear tablas SQL:** Tendrá que crear ocho tablas usando el comando **CREATE TABLE** de SQL, siguiendo estrictamente los siguientes nombres y tipos de datos (en caso de no seguirlos, su solución estará incorrecta debido a que será probada con test automáticos).

products	
product_id	TEXT
product_category_name	TEXT
product_name_length	INTEGER
product_description_length	INTEGER
product_photos_qty	INTEGER
product_weight_g	REAL
product_length_cm	REAL
product_height_cm	REAL
product_width_cm	REAL

orders	
order_id	TEXT
customer_id	TEXT
order_status	TEXT
order_purchase_timestamp	TEXT
order_approved_at	TEXT
order_delivered_carrier_date	TEXT
order_delivered_customer_date	TEXT
order_estimated_delivery_date	TEXT

order_items	
order_id	TEXT
order_item_id	INTEGER
product_id	TEXT
seller_id	TEXT
shipping_limit_date	TEXT
price	REAL
freight_value	REAL

order_reviews	
review_id	TEXT
order_id	TEXT
review_score	INTEGER
review_comment_title	TEXT
review_comment_message	TEXT
review_creation_date	TEXT
review_answer_timestamp	TEXT

customers	
customer_id	TEXT
customer_unique_id	TEXT
customer_zip_code_prefix	INTEGER
customer_city	TEXT
customer_state	TEXT

order_payments	
order_id	TEXT
payment_sequential	INTEGER
payment_type	TEXT
payment_installments	INTEGER
payment_value	REAL

sellers	
seller_id	TEXT
seller_zip_code_prefix	INTEGER
seller_city	TEXT
seller_state	TEXT

category_translation	
product_category_name	TEXT
product_category_name_english	TEXT

6. **Insertar valores:** Con el operador **INSERT INTO** de SQL debe poblar la base de datos.

Parte 2: Escritura de consultas (1.25 Puntos)

Ya concluida la parte uno, usted ya cuenta con la base de datos poblada, y ahora debe escribir consultas SQL a la base de datos para extraer información relevante. Específicamente, debe diseñar, escribir y ejecutar las siguientes consultas SQL en base a la descripción que se entrega a continuación.

- **Consulta facil_1.** Encuentre el identificador de la orden (`order_id`), la ciudad del cliente (`customer_city`) y el estado del cliente (`customer_state`). Filtre los resultados para mostrar únicamente las órdenes que ya fueron entregadas (`order_status = 'delivered'`) y que pertenecen a clientes que residen en el estado de Rio Grande do Sul ('RS').
- **Consulta facil_2.** Se necesita revisar los artículos vendidos a un precio exacto. Muestre el identificador de la orden (`order_id`), el identificador del producto (`product_id`), el precio (`price`) y el nombre de la categoría en portugués (`product_category_name`). Extraiga esta información cruzando las tablas correspondientes, pero solo para los artículos cuyo precio sea exactamente de \$151.
- **Consulta intermedio_1.** Calcule la cantidad de órdenes con retraso agrupadas por el estado de residencia de los clientes (`customer_state`). Para identificar qué órdenes están retrasadas, debe encontrar la intersección entre dos conjuntos de datos: las órdenes que ya fueron entregadas (`order_status = 'delivered'`) y las órdenes cuya fecha real de entrega superó a la fecha estimada (`order_delivered_customer_date > order_estimated_delivery_date`). Otorgue a esta columna de conteo el alias `entregas_atrasadas`. Ordene el resultado de mayor a menor según la cantidad de órdenes con retraso.
- **Consulta intermedio_2.** Descubra las rutas logísticas interestatales más frecuentes en la plataforma. Una “ruta” se define por el estado de origen (`seller_state`) y el estado de destino (`customer_state`), excluyendo los envíos locales (donde el origen y el destino son el mismo estado). Muestre ambos estados (nómbrelos `estado_origen` y `estado_destino`), la cantidad de artículos enviados en esa ruta (nombrada `total_articulos_enviados`) y el valor promedio del envío (nombrado `envio_promedio`). Ordene los resultados de mayor a menor según la cantidad de artículos enviados.
- **Consulta dificil_1.** Encuentre qué ciudades de clientes (`customer_city`) tienen un gasto total acumulado mayor a \$500,000, pero considerando única y exclusivamente los pagos realizados con tarjeta de crédito (`credit_card`) en más de 3 cuotas (`payment_installments > 3`). Muestre la ciudad, el gasto total (use el alias `gasto_total`) y cuántos vendedores distintos enviaron productos a esa ciudad (use el alias `vendedores_distintos`). Ordene el resultado de forma descendente en base al gasto total.
- **Consulta dificil_2.** ¿Qué compran los clientes más insatisfechos en las categorías de mayor volumen? Encuentre el nombre en inglés de la categoría de producto (`product_category_name_english`), la cantidad total de artículos vendidos y el precio promedio de dichos artículos, considerando únicamente las órdenes que recibieron una reseña de 1 estrella (`review_score = 1`). Muestre solo aquellas categorías cuya cantidad vendida sea estrictamente mayor al promedio de ventas por categoría dentro de este mismo grupo de órdenes con reseñas de 1 estrella. Llame a sus columnas calculadas `cantidad_vendida` y `precio_promedio`. Por último, ordene los resultados de forma descendente según la cantidad vendida.

En el notebook entregado ya se encuentran variables definidas para almacenar cada query (como `facil_1`, `intermedio_1`, etc). **Importante:** usted debe dejar sus respuestas (esto es, las consultas SQL) en estas variables. Por ejemplo, para la consulta SQL que responda la consulta `facil_1`, esta debe estar como:

```
facil_1= """
        SELECT *
        FROM a;
        """
```

Si escribe su consulta en otra variable, no se revisará esa consulta y no tendrá el puntaje asociado.

Parte 3: Programación con consultas SQL (1.25 Puntos)

En esta parte, usted debe realizar consultas parametrizadas, esto es, completar las funciones en Python para retornar los resultados solicitados usando SQL parametrizado (según lo visto en ayudantía).

1. **Búsqueda de productos por categoría y presupuesto.** En el notebook cuenta con la función:

```
def buscar_productos_por_presupuesto(db_path, categoria_ingles, precio_maximo):  
    pass
```

Esta función debe retornar el `product_id`, `price` y `seller_id` de los artículos que pertenecen a una categoría en inglés específica y cuyo precio sea menor o igual al `precio_maximo`.

2. **Costo de envío por peso para una ciudad de destino.** En el notebook tenemos la función:

```
def costo_envio_por_peso_y_ciudad(db_path, ciudad_destino, peso_minimo_gramos):  
    pass
```

Esta función debe retornar el `order_id`, `product_weight_g` y `freight_value` de los artículos enviados a una ciudad específica cuyo peso sea mayor o igual al `peso_minimo_gramos`.

Parte 4: Diseño de consultas SQL interesantes (1.25 Puntos)

En esta última parte, usted debe proponer dos consultas “interesantes” para extraer información desde el dataset de Olist. Específicamente, en esta parte usted debe proponer dos consultas donde cada consulta debe incluir su enunciado descriptivo en texto (esto es, explicar en palabras qué desea extraer) y su respectiva consulta en el lenguaje SQL. Además, cada consulta debe incorporar **al menos cinco operadores** de la siguiente lista:

- Filtros compuestos complejos (uso combinado de AND, OR).
- Cruce múltiple: Utilizar tres o más tablas mediante JOIN.
- Subconsultas: Una cláusula SELECT dentro de un WHERE, FROM o el SELECT principal.
- Agrupamiento: GROUP BY con al menos una función de agregación (AVG, SUM, MAX, MIN, COUNT).
- Ordenamiento: ORDER BY.
- Operadores de conjuntos: UNION, INTERSECT o EXCEPT.
- Filtrado de grupos: HAVING.
- Predicados avanzados: Uso de operadores de rango, pertenencia o búsqueda de patrones (BETWEEN, IN, LIKE o NOT LIKE).
- Funciones escalares de transformación: Uso de funciones aritméticas (como ABS, ROUND, CEIL, FLOOR) o de texto (como LOWER, UPPER, SUBSTRING).

Una consulta será considerada interesante en base al criterio subjetivo del ayudante corrector (ella/él determinará si la consulta es interesante o no). En caso de ser una consulta arbitraria y no tener sentido según el contexto de los datos, esta respuesta no será evaluada.

Informe de entrega (1 Puntos)

Adicionalmente con su solución (notebook), usted debe entregar un informe (.pdf), en el cual tiene que explicar su respuesta en la parte 2 y parte 4 de la tarea. Específicamente, debe explicar su diseño y solución de al menos cuatro consultas divididas de la siguiente manera:

- Explicar una consulta intermedia de la Parte 2.
- Explicar una consulta difícil de la Parte 2.
- Explicar ambas consultas de la Parte 4.

En total, su informe debe explicar cuatro consultas descomponiendo la consulta entre los distintos operadores.

Evaluación y entrega

El día límite para la entrega de esta tarea será el Jueves 3 de Septiembre a las 23:59 horas. Para ello se utilizará el buzón de Canvas, donde debe subir solamente el archivo T1_número estudiante.ipynb y el informe en informe_número estudiante.pdf.

Uso de IA y evaluación oral

Para las tareas se permitirá hacer uso de IA si el estudiante estima conveniente, haciendo un uso responsable de la tecnología. En particular, se recomienda que la solución entregada sea escrita por el estudiante individualmente (sin uso de IA).

Para la corrección se hará verificación de uso de IA y se hará una evaluación oral a las tareas en que se indique un mayor uso de esta tecnología, con la intención de verificar si el estudiante entiende la solución entregada. Si $\mathbf{T}$ es la nota obtenida en la tarea y $\mathbf{O}$ es la nota obtenida en la evaluación oral, la nota final de la tarea se calculará como $\min\{\mathbf{T}, \mathbf{O}\}$.

Las evaluaciones orales se realizarán una vez entregadas las correcciones iniciales de las tareas.